fundmat
O blog da materia "Fundamentos de Matemáticas"

Arrays en Sage

Publicado o 19/05/2014 por franpena

Python tiene sus propios tipos de datos, entre los que se encuentran las *listas*. Éstas puede servir tanto para crear arrays unidimensionales,

```
sage: a=[66.25, 333, 1, 1234.5] # vector fila 1x4
```

como bidimensionales,

```
sage: a = [[1,2,3],[4,5,6]] # matriz 2x3
sage: a[0][1]               # posicion (1,2) de la matriz
2
```

Las listas también permiten la creación de arrays con elementos heterogéneos:

```
sage: v = [1, "hello", 2/3, sin(x^3)] # respuesta, entre corchetes
[1, 'hello', 2/3, sin(x^3)]
```

En álgebra lineal, es recomendable usar los tipos específicos de Sage para vectores y matrices.

```
sage: v = vector(RDF,[1, 2, 4]) # respuesta, entre paréntesis
(1, 2, 4)
sage: A = matrix(RDF,[[1,2],[3,4]])
      [1 2]
      [3 4]
sage: A = matrix(RDF,2,2,[1,2,3,4]) # misma matriz, dada por filas
```

Sage define diferentes universos para trabajar con números reales. Nosotros preferiremos el RDF por ser que ofrece mejores resultados para el Cálculo Numérico. Si no se indica lo contrario, el vector pertenece al mínimo universo que contiene a todos los elementos:

```
sage: v = vector([1, 2, 4])   # universo minimo, los enteros
sage: base_ring(v)
Integer Ring
sage: v = vector([1/3, 2, 4]) # universo minimo, los racionales
sage: base_ring(v)
Rational Field
sage: v = vector(RDF,[1/3,2,4]) # explicitando el universo
sage: base_ring(v)
Real Double Field
sage: v                       # muestra hasta 16 decimales
(0.3333333333333333, 2.0, 4.0)
```

```
sage: v.n(digits=2)           # muestra los decimales indicados
(0.33, 2.0, 4.0)
sage: w = v.change_ring(QQ)   # cambia el universo base
sage: w
(1/3, 2, 4)
```

Los índices para las componentes de un vector empiezan en cero:

```
sage: v[0]   # primera componente del vector v
sage: A[0,0] # componente (1,1) de la matriz A
```

Obsérvese que, en las listas Python, el doble índice se separa (por ejemplo, `A[1][2]`) mientras que, en matrices de Sage, se junta (por ejemplo, `A[1,2]`). Puede accederse a varios elementos de un vector o matriz:

```
sage: v[0:2] # accede a las posiciones 0,1 del vector
(1.00000000000000, 2.00000000000000)
sage: A[:,1] # accede a la segunda columna de la matriz
[2]
[4]
```

Las dimensiones de los vectores y matrices pueden consultarse:

```
sage: len(v)               # numero de componentes de un vector
sage: A.nrows(), A.ncols() # numero de filas y cols. de una matriz
```

Creación automática de arrays

La notación `[..]` de Sage (ellipsis range) permite crear listas:

```
sage: [0..7]
[0, 1, 2, 3, 4, 5, 6, 7]
sage: [0,2..10]  # inicio, segundo iterante .. fin
[0, 2, 4, 6, 8, 10]
sage: [0,2..10.] # lista de reales (RR)
[0.000000000000000,
2.00000000000000,
4.00000000000000,
6.00000000000000,
8.00000000000000,
10.0000000000000]
sage: vector(RDF, [0..2]) # vector de reales (RDF)
(0.0, 1.0, 2.0)
```

La función `range` de Python también permite crear **secuencias de enteros**.

```
sage: list(range(8))
[0, 1, 2, 3, 4, 5, 6, 7]
sage: list(range(1,8))
[1, 2, 3, 4, 5, 6, 7]
sage: list(range(0,10,2))   # inicio, fin, paso
[0, 2, 4, 6, 8]
```

```
sage: vector(RDF, range(3)) # vector de reales
(0.0, 1.0, 2.0)
```

En estos ejemplos se ha usado `list()` solo para mostrar el resultado como una lista.

Algunos tipos de matrices se pueden crear con un comando:

```
sage: matrix(RDF, 2, 3)       # matriz 2x3 de ceros
sage: matrix.ones(RDF, 3)     # matriz 3x3 de unos
sage: matrix.identity(RDF, 3) # matriz 3x3 identidad
sage: matrix(RDF, 3, 3, 1)    # matriz 3x3 identidad
sage: matrix.diagonal(RDF, [1,2,3]) # matriz con diagonal [1,2,3]
sage: matrix.random(RDF, 3)   # valores pseudoaleatorios en [-1,1]
```

Operaciones básicas y funciones para arrays

Los vectores y matrices se pueden sumar y multiplicar por escalares con las operaciones habituales:

```
sage: v = vector(RDF, [1..3])
sage: w = vector(RDF, [4..6])
sage: v+w
(5.00000000000000, 7.00000000000000, 9.00000000000000)
sage: 2*v
(2.00000000000000, 4.00000000000000, 6.00000000000000)
```

Otra razón para usar en álgebra lineal los tipos de vectores y matrices en vez de listas es que éstas no responden de igual manera a las operaciones básicas:

```
sage: 2*[4,2] # "multiplicacion" escalar-lista
[4, 2, 4, 2]
```

La multiplicación de vectores se define como su producto escalar:

```
sage: v*w               # producto escalar
32.00000000000000
sage: v.dot_product(w)  # idem usando el metodo .dot_product()
32.00000000000000
sage: v.cross_product(w) # producto vectorial
(-3.00000000000000, 6.00000000000000, -3.00000000000000)
```

El resultado de operar dos matrices con $*$ es la **matriz** resultado de su multiplicación matricial:

```
sage: x = matrix(RDF,[[2],[3]]) # matriz columna
sage: A*x                       # producto matricial
[8.00000000000000]
[18.0000000000000]
```

El resultado de operar con $*$ una matriz y un vector es el **vector** resultado de su multiplicación:

```
sage: x = vector(RDF,[2,3]) # vector
sage: A*x                   # producto matriz-vector
(8.00000000000000, 18.0000000000000)
```

Para otro tipo de operaciones es preciso usar bucles o *list comprehensions*:

```
# vector resultado de la division elemento a elemento
sage: w = vector(RDF, [v[i]/w[i] for i in range(len(v))])
(0.250000000000000, 0.400000000000000, 0.500000000000000)
# cambiar toda la fila de una matriz
sage: A[0,:] = vector(RDF, [2*j for j in range(A.ncols())])
```

Como ejemplo, generemos una lista de puntos igualmente espaciados en $[0, 2\pi]$ formando n subintervalos:

```
sage: n = 10
sage: t = [2*pi*k/n for k in (0..n)]
```

Esta forma concisa de generar vectores o lista se prefiere a los bucles explícitos:

```
sage: t = [] # equivalente aunque prolijo
sage: for k in (0..n):
....:     t.append(2*pi*k/n)
```

A continuación, indicamos otras funciones para vectores y matrices:

```
sage: v.norm()               # norma del vector
sage: v.column()             # construye una matriz columna
sage: A.T                    # matriz traspuesta
sage: A.diagonal()           # diagonal
sage: A.rank()               # rango de la matriz
sage: A.det()                # determinante de la matriz
sage: ~A                     # matriz inversa
sage: A.eigenvalues()        # autovalores
sage: A.eigenvectors_right() # autovectores (por la derecha)
```

Se pueden aplicar expresiones simbólicas a vectores y matrices, elemento a elemento:

```
sage: f(x) = sin(2*x)
sage: M = matrix(RDF, 2,2, [1,2,3,4])
sage: M.apply_map(f)
[  0.9092974268256817  -0.7568024953079282]
[-0.27941549819892586   0.9893582466233818]
```

La operación anterior puede realizarse sobre listas de Python con el comando `map`:

```
sage: t = [1..4.]
sage: f(x) = sin(2*x)
sage: list(map(f,t))
```

```
[0.909297426825682, -0.756802495307928, -0.279415498198926,
0.989358246623382]
```

Podemos construir matrices por bloques yuxtaponiendo en filas y columnas otras matrices o valores constantes:

```
sage: A = matrix(QQ, 2, 2, [3,9,6,10]) # matriz 2x2
sage: matrix.block([[A, -A], [~A, 0]]) # matriz 4x4
[    3     9|   -3   -9]
[    6    10|   -6  -10]
[-----------+-----------]
[-5/12   3/8|    0     0]
[  1/4  -1/8|    0     0]
```

Resolución de sistemas lineales

Un sistema linear $Ax=b$ se resuelve con el operador \:

```
sage: A = matrix(RDF,2,2,[1..4])
sage: b = vector(RDF,[1,1])
sage: x = A.solve_right(b)
(-1.0, 1.0)
```

Esta entrada foi publicada en Sage. Ligazón permanente.

2 Responses to *Arrays en Sage*

Pingback: *Factorización en Sage | fundmat*

Pingback: *Factorización de matrices en Sage | sageformaths*

fundmat

Crea un sitio web ou un blogue de balde en WordPress.com.